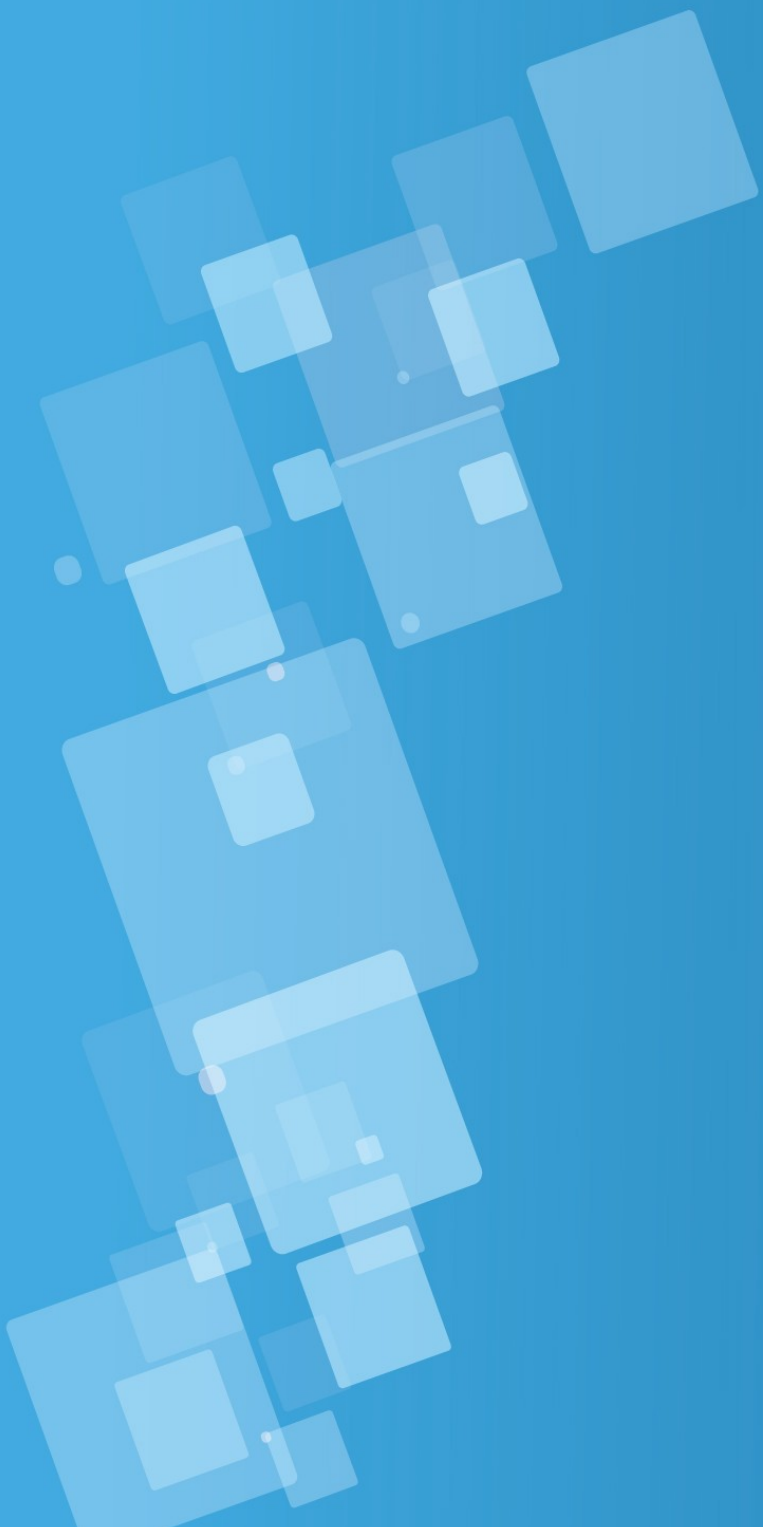




Técnicas Digitales II

NVIC (Nested Vectored Interrupt Controller)

- Bibliografía

Computer Organization and Design ARM Edition, 2017
Capítulo 4.9

The Definitive Guide to ARM Cortex-M3 and Cortex-M4
Processors 3ra Edición de Joseph Yiu
Capitulo 7

- Soporte

Harris & Harris. Digital design and computer architecture:
ARM edition. Elsevier, 2015. Capítulo 6.



El NVIC (Nested Vectored Interrupt Controller)

¿Qué es el NVIC?

- Es un bloque de hardware especializado e integrado dentro del núcleo Cortex-M que centraliza la gestión de todas las excepciones e interrupciones del sistema.



¿ Qué es una Excepción ?

- Es un evento no programado, distinto a un salto o una llamada a función y que cambia el flujo normal de ejecución de instrucciones.
- Son como funciones pero se llaman de manera no programada o en un momento no previsto.
- Pueden ser causadas por el *Hardware* o el *Software*.

No debe confundirse con el manejo de excepciones que realizan los lenguajes de alto nivel para la administración de errores.



¿ Qué es una Excepción ?

- Es una de las parte mas importante en el diseño de un microprocesador.
- Inicios
 - En un principio fueron utilizadas para gestionar eventos dentro del procesador (como desbordamiento de coma flotante o división por cero)
*Estas excepciones son llamadas **Internas***
 - Luego se extiende su uso para el manejo y comunicación con dispositivos de E/S
*Estas excepciones son llamadas **Externas***



Clasificación según Origen y Sincronía

- Sincronía:
 - **Sincrónicas:** Ocurren en el mismo punto del programa cada vez que se ejecuta (ej. división por cero, instrucción indefinida).
 - **Asincrónicas:** Ocurren de forma impredecible respecto al flujo del programa (ej. periféricos de I/O, fallos de hardware).
- Origen:
 - **Internas:** Generadas por la propia lógica del procesador.
 - **Externas:** Generadas por eventos fuera de la CPU.



Casos típicos

- **I/O Device Request:** Externa y Asincrónica (Interrupción). Provocada por dispositivos de I/O como un teclado, un ADC, etc.
- **Invoke OS Service (Trap/Syscall):** Interna y Sincrónica (Invocación voluntaria).
- **Undefined Instruction:** Interna y Sincrónica (Error de código).
- **Hardware Malfunction:** Externa y Asincrónica (Falla física).
- **Arithmetic Overflow:** Interna y Sincrónica (Error de cómputo).



¿Qué sucede dentro del Datapath?

Cuando ocurre una excepción, el hardware debe realizar tres acciones críticas de forma atómica:

- 1. Guardar el PC:** La dirección de la instrucción interrumpida se almacena en el ELR (Exception Link Register).
- 2. Registrar la Causa:** Se guarda un código que identifica el motivo en el SPSR o un registro de estado de excepción.
- 3. Salto al Manejador:** El hardware fuerza al Program Counter (PC) a saltar a una dirección de memoria predefinida donde reside el código de gestión de la excepción.



¿Cómo sabe el software qué pasó?

Tenemos dos métodos principales para que el manejador identifique la excepción:

- **Método 1:** Registro de Causa (Polled): El procesador tiene un único punto de entrada para todas las excepciones. El software lee un registro de estado para saber qué pasó y salta a la rutina correspondiente.
- **Método 2:** Interrupciones Vectorizadas: El hardware salta directamente a una dirección diferente para cada causa, según una Tabla de Vectores.



El costo del imprevisto

- **Dificultad en el Pipeline:** Las excepciones complican el diseño de procesadores modernos porque pueden ocurrir en cualquier etapa del pipeline (Fetch, Decode, etc.).
- **Excepciones Precisas:** Un procesador debe garantizar que todas las instrucciones previas a la excepción se hayan completado y que las posteriores no hayan cambiado el estado del procesador.

En el manejo de excepciones tanto Hardware como el Software de Sistema (Kernel) deben garantizar la robustez y seguridad del microprocesador.



Interfaz Software / Hardware

¿ Qué implica que Hardware y el Software de Sistema (Kernel) deban garantizar la robustez y seguridad del computador ?.

1. Responsabilidad del Hardware

- El flujo de control detecta el evento (ej. una instrucción no definida).
- Se detienen las instrucciones en vuelo dentro del pipeline.
- Guarda el Estado:
 - Dirección de la instrucción interrumpida en el ELR (Exception Link Register).
 - Motivo en el registro de Causa (o ESR - Exception Syndrome Register).
- Salto de Control: Carga en el PC la dirección del manejador de excepciones (Vector de interrupción).



Interfaz Software / Hardware

¿ Qué implica que Hardware y el Software de Sistema (Kernel) deban garantizar la robustez y seguridad del computador ?.

2. Responsabilidad del Software (Manejador/OS)

- Preservación de Contexto: Guarda en la pila (Stack) los registros que el hardware no salvó y que el manejador va a utilizar.
- Identificación y Acción: Lee el registro de causa, decide qué rutina ejecutar (ej. cerrar un programa o atender un teclado) y realiza la tarea.
- Restauración y Retorno: Recupera los registros de la pila y ejecuta la instrucción de retorno (ERET en v8 o BX LR en nuestra Blue Pill).



Llamado

- En resumen, una excepción es similar a una función
- Tendremos aspectos similares y otros que se suman.



Llamado

Similar a una función estándar

- Debe guardar la dirección de retorno.
- Saltar a la dirección del código que atiende a la excepción.
- Restaura a los valores antes de retornar al código.
- Retornar a la dirección desde donde fue llamada.



Llamado

Diferencias o aspectos que se suman a una función estándar

- No hay una instrucción con la dirección de salto, el salto está determinado por una tabla de vectores con las direcciones de las funciones que atienden a las excepciones.
- En el momento de llamada, ciertos registros claves son reemplazados para mantener los valores originales intactos.
- Pueden cambiar los privilegios, permitiendo a las excepciones acceder a partes protegidas de la memoria.

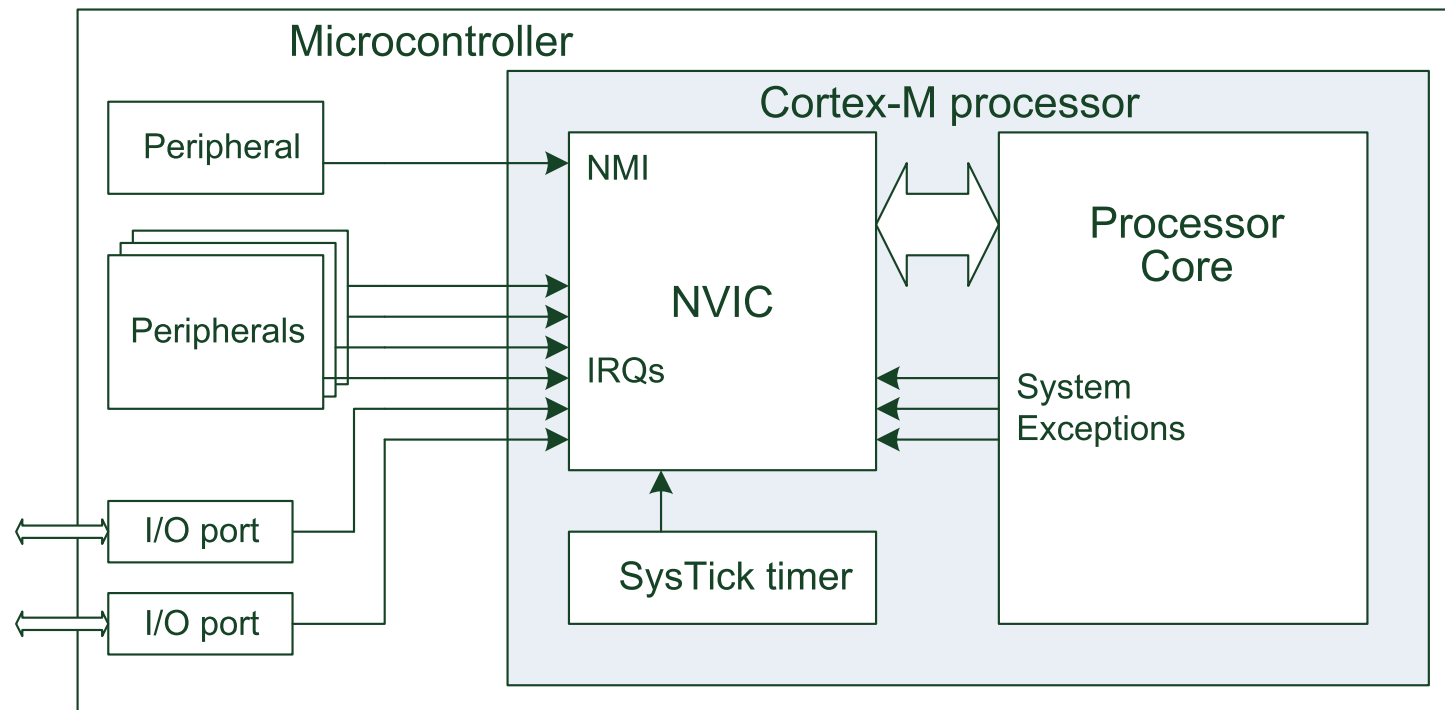


NVIC: El Gestor de Eventos en Tiempo Real

- Vimos entonces:
 - Lo que era una excepción y sus similitudes con una función.
 - Que el NVIC era el controlador de excepciones en el Cortex-M
- ¿ Cómo hacemos para utilizar este periférico y poder construir nuestras interrupciones ?

¿Que es el "NVIC"?

- **Nested Vectored Interrupt Controller.**
- Es el controlador de todos los Cortex-M3, es decir:
 - No depende del fabricante y por lo tanto es un periférico interno del núcleo ARM Cortex-M.
- Función Principal: Decidir qué interrupción se atiende, cuándo y con qué prioridad.





¿Que significa NVIC (Nested Vectored Interrupt Controller) ?

- **Nested** (Anidado): Una interrupción de alta prioridad puede interrumpir a otra de menor prioridad que ya esté en ejecución.
Esta capacidad se denomina “*Pre-emption*” o desalojo
- **Vectored** (Vectorizado): El hardware salta directamente a la dirección de memoria donde está la función de servicio (ISR), sin necesidad de buscarla por software.



El NVIC en el Ecosistema ARMv7-M

- **Ubicación:** Está estrechamente acoplado al núcleo para lograr una latencia mínima.
- **Determinismo:** El tiempo que tarda en entrar a una interrupción es fijo (12 ciclos de reloj), algo muy importante para sistemas de RT.
- **Soporte:** Gestiona tanto las excepciones internas del núcleo (Reset, NMI, HardFault) como las interrupciones externas de los periféricos (Timers, UART, ADC).



Numeración de Excepciones

- Del 1 al 15 corresponde a excepciones del sistema, el 0 no existe

Ex.Nro	Tipo de Ex.	Pr.	Descripción
1	Reset	-3	Reinicio
2	NMI	-2	Interrupción no enmascarable, generado por periféricos o del exterior
3	Hard Fault	-1	Todas las condiciones de falla, si el controlador de falla correspondiente no está habilitado
4	MemManFault	Prog	Violación de MPU o ejecución de código desde memoria no permitida
5	Bus Fault	Prog	El AHB recibe una respuesta de error de un esclavo
6	Usage Fault	Prog	Error al intentar acceder al coprocesador (M3 y M4 no tienen coprocesador)
7-10	Reservado	N/A	
11	SVC	Prog	Llamada de supervisor, generalmente utilizada para acceder a serv. del SO
12	Debug Monitor	Prog	Utilizada en Debug, generalmente para eventos como breakpoint
13	Reservado	N/A	
14	PendSV	Prog	Excepción utilizada en general por SO para cambios de contexto
15	SYSTICK	Prog	Excepción generada por el timer incluido en el procesador, utilizado como temporizador por el SO



Numeración de Excepciones

- Del 16 en adelante son las entradas de interrupciones, estas son generadas por periféricos internos al chip (en caso de microcontroladores o SoC), o por interrupciones externas.

El orden y tipo lo define el fabricante del chip

Ex.Nro	Tipo de Ex.	Pr.	Descripción
16	Interrupción #0	Prog	Se pueden generar a partir de periféricos en el chip o de fuentes externas
17	Interrupción #1	Prog	
...			
255	Interrupción #239	Prog	



Numeración CMSIS

Importante: si usamos CMSIS, la numeración cambia y se utiliza un enumerado (IRQn_Type) donde:

- Excepciones de Sistema: Tienen valores negativos.
 - Reset_IRQn = -15
 - SysTick_IRQn = -1
- Interrupciones Externas (IRQs): Comienzan en 0 de forma ascendente.
 - WWDG_IRQn = 0
 - RTC_IRQn = 3

Numeración CMSIS

- Tabla de la numeración según CMSIS

Ex.Nro	Tipo de Ex.	CMSIS-Core	CMSIS-Core Enum	Nombre del manejador de excepción
1	Reset			Reset_Handler
2	NMI	NonMaskableInt_IRQn	-14	NMI_Handler
3	Hard Fault	HardFault_IRQn	-13	HardFault_Handler
4	MemManFault	MemoryManagement_IRQn	-12	MemManage_Handler
5	Bus Fault	BusFault_IRQn	-11	BusFault_Handler
6	Usage Fault	UsageFault_IRQn	-10	UsageFault_Handler
11	SVC	SVC_IRQn	-5	SVC_Handler
12	Debug Monitor	DebugMonitor_IRQn	-4	DebugMon_Handler
14	PendSV	PendSV_IRQn	-2	PendSV_Handler
15	SYSTICK	SysTick_IRQn	-1	SysTick_Handler
16	Interrup #0	(especificación del disp)	0	(especificación del disp)
17-255	Interrup #1-239	(especificación del disp)	1-239	(especificación del disp)



Gestión de las Interrupciones

- Cortex-M posee una serie de registros para gestionar las interrupciones
- La mayoría en NVIC pero algunos en la *System Control Block (SCB)*
- Hay también registros especiales dentro del núcleo para enmascarar interrupciones
- Finalmente CMSIS-Core provee de funciones que facilitan el proceso de configuración de las interrupciones
 - La desventaja del acceso directo a los registros en lugar de CMSIS es disminuir la portabilidad a otros Cortex



Gestión de las Interrupciones

- Las principales funciones de CMSIS para gestionar las interrupciones son

Función	Uso
void NVIC_EnableIRQ (IRQn_Type IRQn)	Habilitar una interrupción externa
void NVIC_DisableIRQ (IRQn_Type IRQn)	Deshabilitar una interrupción externa
void NVIC_SetPriority (IRQn_Type IRQn, uint32_t priority)	Configurar la prioridad de una interrupción
void __enable_irq(void)	Borrar PRIMASK para habilitar interrupciones
void __disable_irq(void)	Configurar PRIMASK para deshabilitar interrupciones
void NVIC_SetPriorityGrouping(uint32_t PriorityGroup)	Establecer la configuración de grupo y sub-prioridad



Gestión de las Interrupciones

Luego de un reset, todas las interrupciones están deshabilitadas y con prioridad 0

El proceso para habilitar una interrupción es:

- Configurar el Nivel de prioridad
- Habilitar el control de interrupciones en el periférico
- Habilitar la Interrupción en el NVIC
- Finalmente al activarse la interrupción se ejecuta la ISR (*Interrupt Service Routine*) correspondiente
 - La dirección de esta rutina se encuentra en el vector correspondiente a la interrupción



Definiciones de prioridad

- Aceptar una excepción y ejecutar su ISR, puede depender de la prioridad de la excepción y de la prioridad del procesador
- Una excepción de mayor prioridad (menor número) puede adelantarse a una de menor prioridad.
- Cortex-M3 y M4 tienen 3 excepciones con prioridad fija (NMI, Reset y HardFault) y las otras con 256 niveles programables.
- Por motivos de complejidad, los 256 niveles no suele estar en las implementaciones, reduciendo a 8, 16 o 32 niveles.
- Además y por tener al menos 1 nivel de sub-prioridad, los niveles de Pre-emption no serán superiores a 128

Definiciones de prioridad

- Para configurar los niveles, se utiliza un registro de nivel de prioridad
- La cantidad de niveles de prioridad implementado dependerá de la cantidad de bits útiles de este registro (del bit mas significativo al menos)
- Ejemplo de caso con 3 bit para la configuración de nivel

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Implementado			No implementado				

Aquí los niveles posibles son 8: 0x20,0x40,0x80,0x60, etc

Empezar de arriba hacia abajo, facilita portar el software

Definiciones de prioridad

- Hablamos de 3 interrupciones que eran fijas, estas tiene la mayor prioridad, y como mayor prioridad es menor número, se establecen como números negativos
- Como ejemplo y dependiendo de los bit implementados, tenemos

Nivel de Prioridad	Tipo de excepción	Implementación de 3-bit	Implementación de 5-bit	Implementación de 8-bit
-3	Reset	-3	-3	-3
-2	NMI	-2	-2	-2
-1	Hard Fault	-1	-1	-1
0	Excepciones con prioridad programable	0x00	0x00	0x00
1		0x20	0x08	0x01
2		0x40	0x10	0x02



Definiciones de prioridad

- Si bien tenemos un registro de 8 bits (256 combinaciones), solo se dispone como máximo de 128 niveles de prioridad “pre-emption”
- Esto es porque el registro de prioridad se divide en 2 campos

Grupo o pre-emption: Define que evento tiene mayor prioridad. Si mientras se atiende un evento ocurre otro de mayor prioridad, se detiene el actual y se atiende al de mayor prioridad “pre-emption”

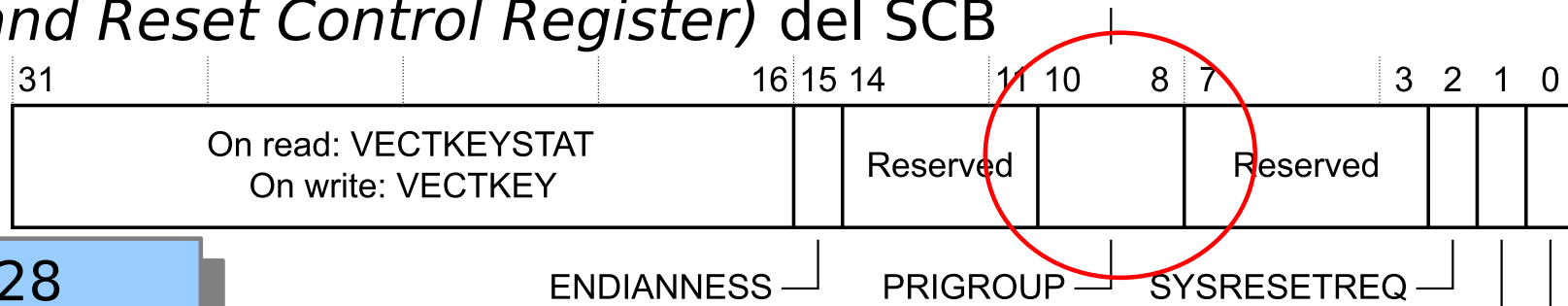
Bits mas altos, hasta 7 bits para los 128 niveles máximos.

Sub-prioridad: Si dos eventos de la misma prioridad ocurren de manera SIMULTÁNEA, el evento con mayor “sub-prioridad” se atiende primero, dejando al otro en espera.

Bit menos significativos, de al menos 1 bit.

Configuración de Grupo y Sub-prioridad

Los bits utilizados para cada campo son configurables mediante el campo **PRIGROUP** del registro AIRCR (*Application Interrupt and Reset Control Register*) del SCB



128
niveles

PRIGROUP	Binary point ^a	Group priority bits	Subpriority bits	Group priorities	Subpriorities
0b000	bxxxxxxx.y	[7:1]	[0]	128	2
0b001	bxxxxxx.yy	[7:2]	[1:0]	64	4
0b010	bxxxxx.yyy	[7:3]	[2:0]	32	8
0b011	bxxxx.yyyy	[7:4]	[3:0]	16	16
0b100	bxxx.yyyyy	[7:5]	[4:0]	8	32
0b101	bxx.yyyyyy	[7:6]	[5:0]	4	64
0b110	bx.yyyyyyy	[7]	[6:0]	2	128
0b111	b.yyyyyyyy	None	[7:0]	1	256

a. PRI_n[7:0] field showing the binary point. x denotes a group priority field bit, and y denotes a subpriority field bit.

Sin
grupos



Configuración de Grupo y Sub-prioridad

Para evitar el uso de registros, CMSIS provee de funciones para la manipulación del grupo y sub-prioridad.

Función	Uso
void NVIC_SetPriorityGrouping(uint32_t PriorityGroup)	La vimos anteriormente, configura el PRIGROUP
uint32_t NVIC_EncodePriority (uint32_t PriorityGroup, uint32_t Pre emptPriority, uint32_t Sub priority)	Permite codificar la prioridad en base a la configuración del grupo y el grupo y sub-prioridad requerida.
void NVIC_DecodePriority (uint32_t Priority, uint32_t PriorityGroup, uint32_t *pPre emptPriority, uint32_t *pSub priority)	Realiza el proceso inverso, dada un valor de prioridad y una configuración de grupo, devuelve el grupo y sub-prioridad

Finalmente con **NVIC_SetPriority()** establecemos la prioridad en una determinada interrupción

Configuración de los grupos y la implementación

- Si utilizamos sub-prioridades, debemos configurar PRIGROUP
 - **IMPORTANTE:** tener en cuenta los bits que tiene implementado el procesador.
- El registro de prioridad se divide en 2 campos, pero los bits que están fuera de la implementación siempre serán 0.
- Ejemplo:
En un procesador con 3 bit de prioridad habilitados y un grupo de prioridad configurado en 0x1 (bxxxxxx.yy), tenemos:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Grupo o pre-emption						sub-prioridad	
Implementado			Siempre en 0				

Es decir, en realidad tenemos, 8 niveles de grupo y 0 de sub-prioridad.

La Tabla de Vectores

- Cuando el Cortex-M acepta una solicitud de excepción, debe determinar la dirección del ISR
- Esta información se almacena en una tabla de vectores en la memoria.
- Esta tabla arranca por defecto en la dirección 0x0000 0000
- Adicionalmente esta tabla posee en la 1era dirección, el valor inicial del puntero del Stack (MSP)

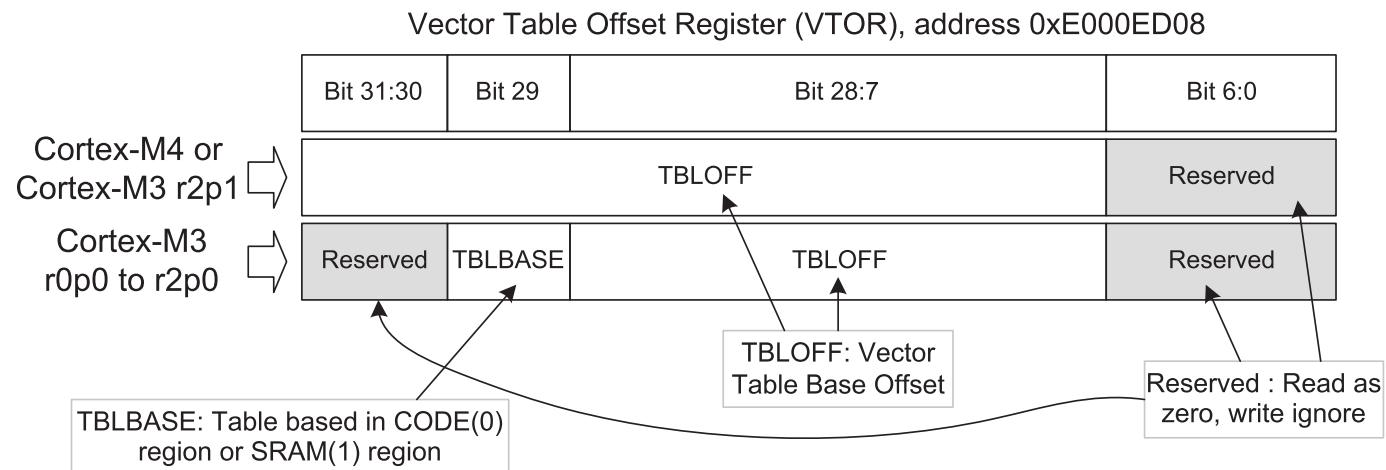
Memory Address		Exception Number
0x0000004C	Interrupt#3 vector	19
0x00000048	Interrupt#2 vector	18
0x00000044	Interrupt#1 vector	17
0x00000040	Interrupt#0 vector	16
0x0000003C	SysTick vector	15
0x00000038	PendSV vector	14
0x00000034	Not used	13
0x00000030	Debug Monitor vector	12
0x0000002C	SVC vector	11
0x00000028	Not used	10
0x00000024	Not used	9
0x00000020	Not used	8
0x0000001C	Not used	7
0x00000018	Usage Fault vector	6
0x00000014	Bus Fault vector	5
0x00000010	MemManage vector	4
0x0000000C	HardFault vector	3
0x00000008	NMI vector	2
0x00000004	Reset vector	1
0x00000000	MSP initial value	0

Note : LSB of each vector must be set to 1 to indicate Thumb state

La Tabla de Vectores - Reubicación

La dirección por defecto de arranque, corresponde en general a FLASH o ROM.

- **Desventaja:** no se puede modificar mientras el procesador funciona.
- **Alternativa:** reubicar la tabla a otra posición (RAM)
 - Esto se realiza por medio de **SCB->VTOR**



La Tabla de Vectores - Reubicación

- Ejemplo donde se mueve el vector a la SRAM

```
#define HW32_REG(ADDRESS) (*((volatile unsigned long *)(ADDRESS)))
#define VTOR_NEW_ADDR 0x20000000

// Copiar el vector original a la SRAM antes de modificar VTOR
int i;
for (i=0;i<48;i++){ // copia hasta el vector 48
    HW32_REG((VTOR_NEW_ADDR + (i<<2))) = HW32_REG((i<<2));
}
__DMB(); // Data Memory Barrier
        // Asegurar que finalizó la esc. de mem)

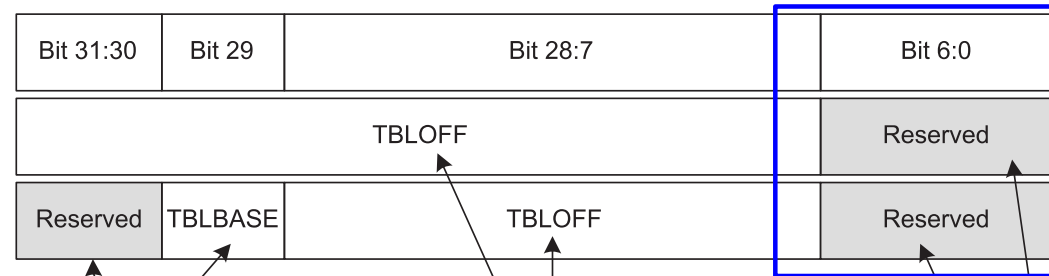
SCB->VTOR = VTOR_NEW_ADDR; // Colocar el nuevo vector
__DSB(); // Data Synchronization Barrier
        // Espera que todas las escritura finalicen
```

La Tabla de Vectores - Reubicación

Ojo con la alineación

- No todas las direcciones son posibles
- La dirección de arranque, deben estar alineados a la potencia de 2 posterior a la longitud del vector
- Ejemplo
 - La bluepill tiene implementada 43 IRQs externas.
 - 16 Internas + 43 Externas = 59
 - Longitud $59 \times 4 = 236$ bytes
 - La potencia de 2 siguiente a 236 es 256
 - El vector debe estar alineado a 256 es decir los 8 bits mas bajo son 0 quedando los valores posibles en **0xFFFF.XX00**

Vector Table Offset Register (VTOR), address 0xE000ED08





Entradas de Interrupción y Estados

Una interrupción puede estar en uno de estos estados:

- **Deshabilitada (Disable)**: estado por defecto no está habilitada para generar interrupción o **Habilitada (Enable)**: Habilitada para generar interrupción
- **Pendiente (Pending)**: Solicitada pero esperando ser atendida por el procesador.
- **Activa (Active)**: Siendo ejecutada actualmente



Entradas de Interrupción y Estados

Hay combinaciones de estos estados:

Ejemplo: **Temporizador**

- **Estado Activo (Active):** Ocurre el evento del temporizador y el procesador salta a la ISR, NVIC marca la interrupción como Activa.
- **Deshabilitar (Disable):** Dentro del código de esa ISR, se deshabilita esa interrupción (la ISR sigue ejecutándose).
- **Nueva petición (Pending):** Mientras la ISR se ejecuta, el periférico genera otra señal de interrupción, el NVIC detecta la señal, pone el bit de Pendiente (Pending) a '1'.

Estado resultante

- **Activa:** El procesador todavía no ha salido de la función (ISR).
- **Deshabilitada:** Porque el software le quitó el permiso.
- **Pendiente:** Llegó una nueva solicitud.

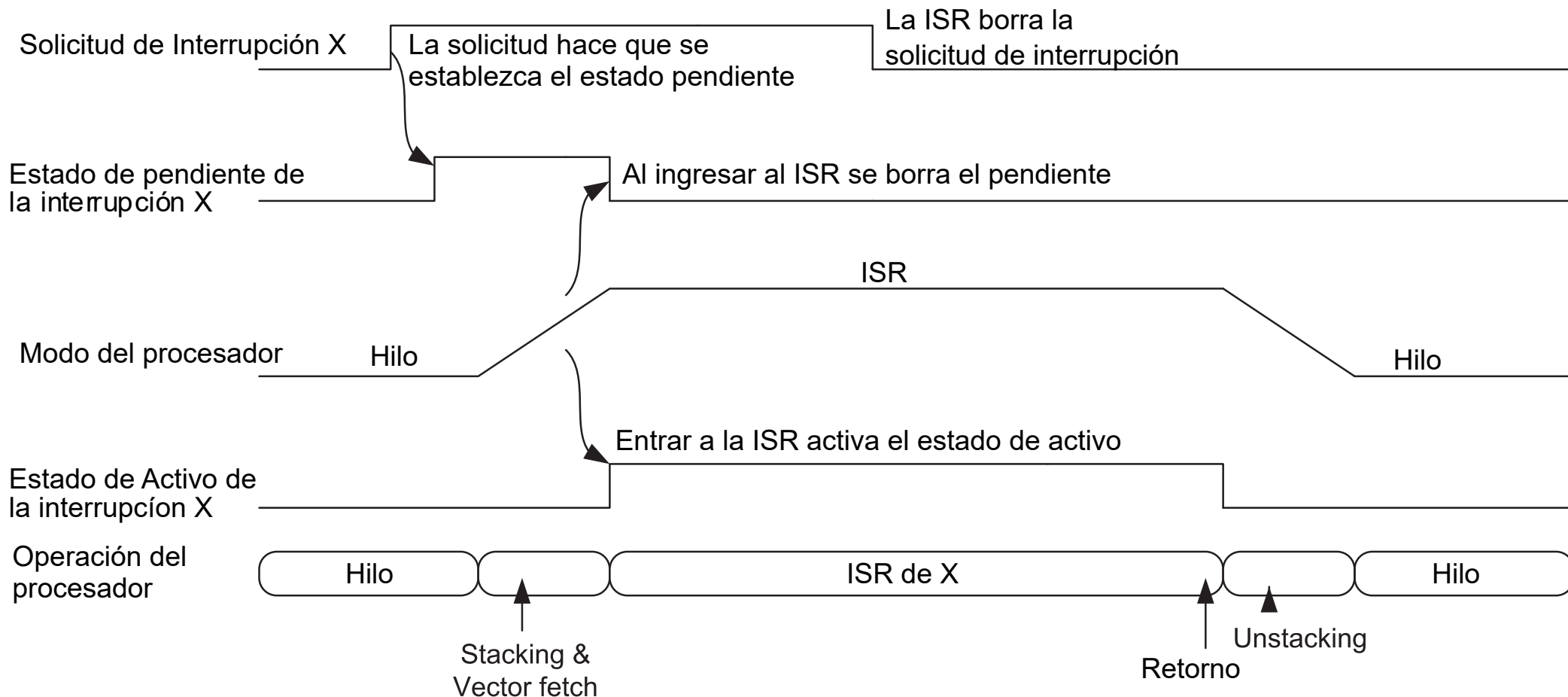


Entradas de Interrupción y Estados

La condición para que una interrupción sea procesada

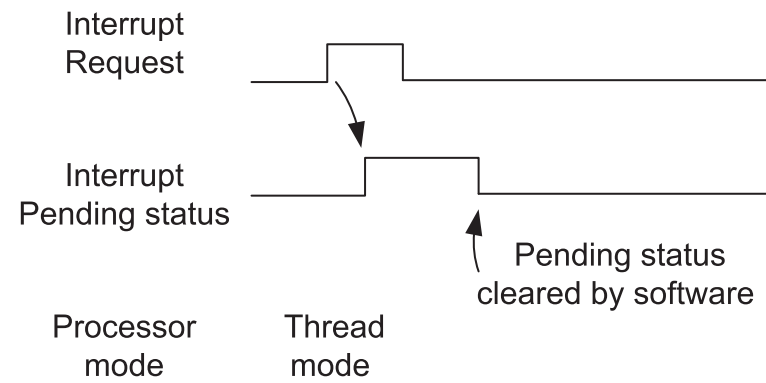
- **Estado Pendiente**
- **Interrupción Habilitada**
- **La prioridad de la interrupción es mayor que el nivel actual** (incluida la configuración del registro de enmascaramiento de interrupciones).
 - **Nivel Actual:** si está dentro del programa principal es el mas bajo, si está dentro de una ISR es el nivel de la ISR
 - **Registro de Enmascaramiento:** se puede artificialmente subir la prioridad actual, para que interrupciones de menor prioridad no se ejecuten

Estados al generar una solicitud de interrupción

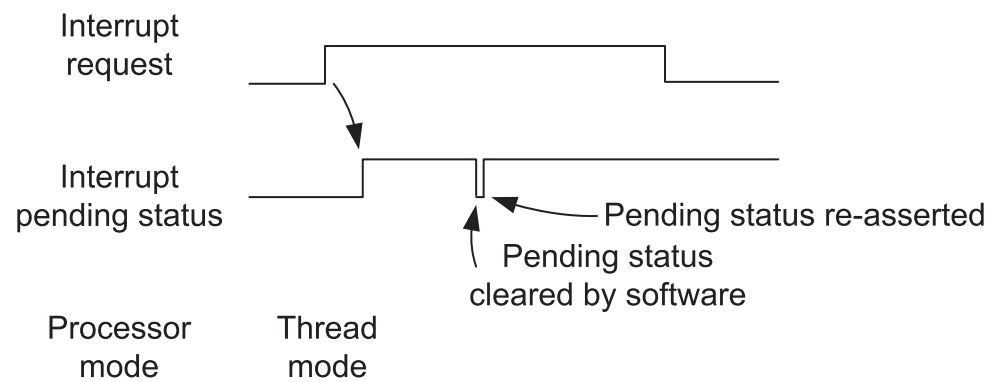


Estados al generar una solicitud de interrupción

- Si el estado pendiente se borra antes de ser atendida, la interrupción nunca se ejecuta.

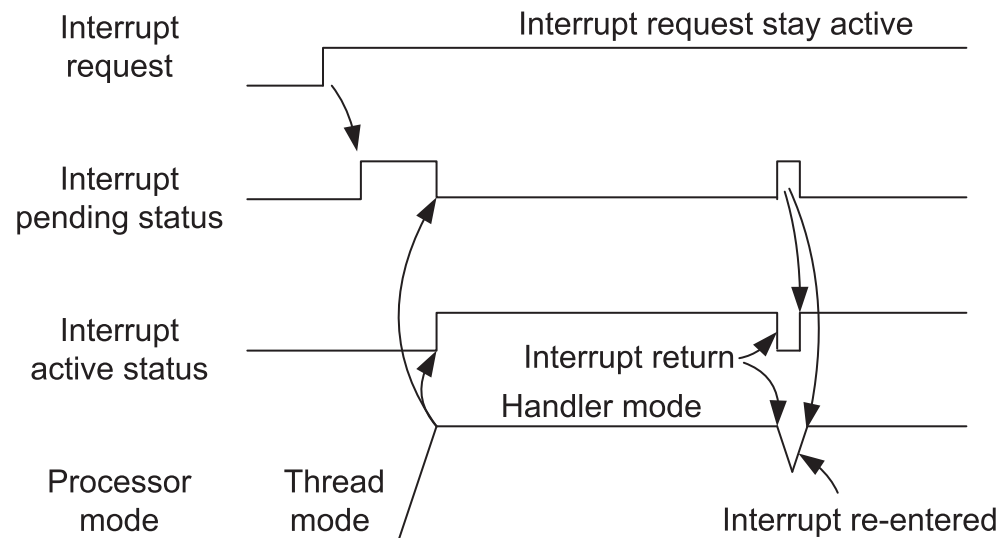


- Si la solicitud de una interrupción no se borra, al borrar el estado pendiente se vuelve a re-activar el mismo inmediatamente



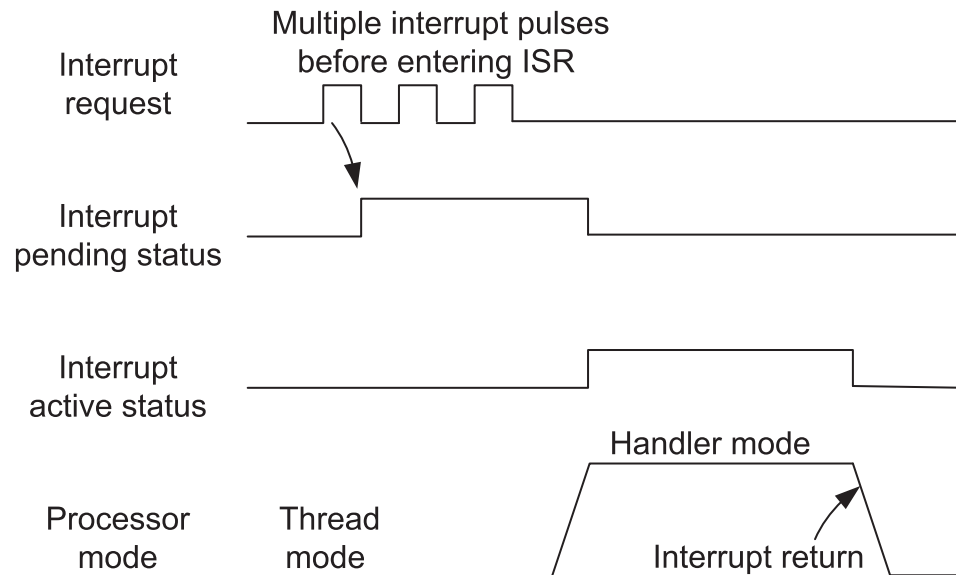
Estados al generar una solicitud de interrupción

- Si la solicitud de una interrupción no se borra, al finalizar el ISR, se volverá a poner en pendiente y se llamará nuevamente.



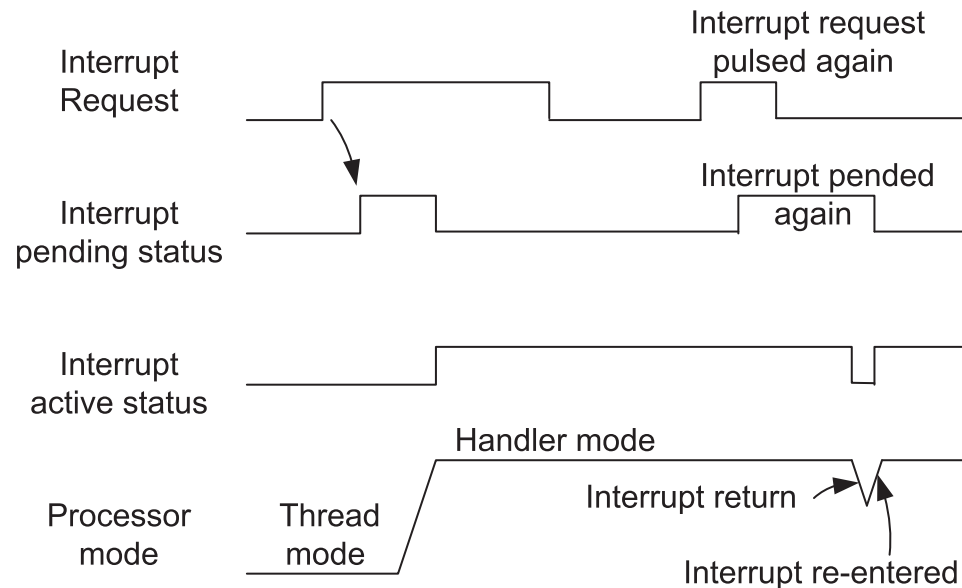
Estados al generar una solicitud de interrupción

- Si se realiza varias solicitudes antes de que una interrupción sea atendida, se toma como una sola solicitud.



Estados al generar una solicitud de interrupción

- Si se realiza una solicitud cuando la interrupción está activa, la interrupción queda como pendiente y al finalizar el ISR es llamada de nuevo.





Secuencia de entrada (Exception Entrance)

Una vez aceptada, el hardware realiza automáticamente varias operaciones para minimizar la latencia:

- Stacking (Apilado): Se guardan ocho registros en el stack actual (MSP o PSP): R0-R3, R12, LR, PC y xPSR. Esto permite que el manejador se escriba como una función de C normal.
- Vector Fetch (Búsqueda del vector): Se obtiene la dirección de inicio del ISR desde la tabla de vectores, operación que puede ocurrir en paralelo al stacking.
- Se comienza a cargar las instrucciones
- Actualización de registros del NVIC (estado de pendiente, activo) y del procesador (PSR, LR, PC y SP)
 - El PC lo hará con la dirección de inicio del ISR
 - El LR lo hará con el valor EXC_RETURN de 32 bits (27 bit altos en 1 y el resto con información de la interrupción)



Ejecución del ISR

Durante la ejecución de la Rutina de Servicio de Interrupción (ISR), el procesador entra en Handler Mode.

En este modo :

- Se utiliza obligatoriamente el Main Stack Pointer (MSP) para las operaciones de pila.
- La ejecución es siempre en nivel de acceso privilegiado.

Si llega una excepción:

- **Mayor prioridad:** la actual se suspende (nested exception).
- **Igual o menor:** se queda en estado pendiente.



Retorno de la excepción (Exception Return)

En lugar de utilizar una instrucción especial para el retorno, se utiliza un mecanismo que evita una codificación especial para las ISR en C

Procedimiento: Se carga el valor EXC_RETURN en el PC mediante instrucciones como BX LR, POP {PC} o LDR/LDM al PC.

Cuando el procesador, detecta este valor en el PC:

- Se restaura los valores de los registros guardados en el stack hacia el banco de registros del procesador.
- Se actualizan los registros de estado del NVIC (limpiando el bit de activo) y los registros internos del core (como el PSR), regresando al flujo del programa original.

Registros del NVIC

Dirección	Registro	CMSIS	Función
0xE000E100 a 0xE000E11C	Habilitar Interrupción	NVIC->ISER [0..7]	Escribir 1 para habilitar
0xE000E180 a 0xE000E19C	Deshabilitar Interrupción	NVIC->ICER [0..7]	Escribir 1 para deshabilitar
0xE000E200 a 0xE000E21C	Establecer como pendiente	NVIC->ISPR [0..7]	Escribir 1 para pendiente
0xE000E280 a 0xE000E29C	Borrar pendiente	NVIC->ICPR [0..7]	Escribir 1 para borrar pendiente
0xE000E300 a 0xE000E31C	Leer estado de activo	NVIC->IABR [0..7]	1 indica activo, solo lectura
0xE000E400 a 0xE000E4EF	Registros de prioridad	NVIC->IPR [0..] to NVIC->IPR [59]	8 bits de prioridad para cada interrupción (cada registro de 32 bits tiene 4 registros de 8)
0xE000EF00	Disparador de interrupciones	NVIC->STIR	Escribe la interrupción a disparar



Registros del NVIC

Registros de Habilitación (ISER e ICER)

El NVIC utiliza dos tipos de registros para controlar el permiso de las interrupciones, diseñados para evitar operaciones de lectura-modificación-escritura que podrían causar condiciones de carrera:

- **ISER (Interrupt Set-Enable Registers):** '1' habilita la interrupción correspondiente. '0' no tiene efecto.
0xE000E100 a 0xE000E11C o NVIC->ISER[0..7]
- **ICER (Interrupt Clear-Enable Registers):** '1' deshabilita la interrupción. '0' no tiene efecto.
0xE000E180 a 0xE000E19C o NVIC->ICER[0..7]

```
void NVIC_EnableIRQ (IRQn_Type IRQn); // Habilita la interrupción
void NVIC_DisableIRQ (IRQn_Type IRQn); // Deshabilita la interrupción
```

Registros del NVIC

Registros de Pendiente (ISPR e ICPR)

Permiten al software manipular manualmente el estado de "espera" de una interrupción:

- **ISPR (Interrupt Set-Pending Registers):** Se usa para forzar una interrupción por software. Escribir un '1' pone la interrupción en estado pendiente.
0xE000E200 a 0xE000E21C o NVIC->ISPR [0..7]
- **ICPR (Interrupt Clear-Pending Registers):** Se usa para cancelar una interrupción que está pendiente pero aún no ha sido atendida. Escribir un '1' limpia el estado pendiente.
0xE000E280 a 0xE000E29C o NVIC->ICPR [0..7]

```
void NVIC_SetPendingIRQ(IRQn_Type IRQn);    // conf. como pendiente
void NVIC_ClearPendingIRQ(IRQn_Type IRQn);  // borra pendiente
uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn); // lee estado de pendiente
```



Registros del NVIC

Registro de Estado Activo (IABR)

- Es un registro de solo lectura que indica qué interrupciones están siendo ejecutadas actualmente por el procesador.
- Un bit en '1' significa que el manejador de esa interrupción está corriendo o fue interrumpido por una excepción de mayor prioridad (anidamiento).
- Útil para herramientas de depuración (debuggers) para visualizar el estado real del sistema.

0xE000E300 a 0xE000E31C o NVIC->IABR[0..7]

```
uint32_t NVIC_GetActive(IRQn_Type IRQn); // devuelve estado de activo
```

Registros del NVIC

Registros de Prioridad (IPR)

- Cada interrupción externa tiene asignado un campo de prioridad en estos registros.
- Se define 8 bits para la prioridad, en general solo se implementan los bits más significativos (MSB), (3 o 4 bits)→(8 o 16 niveles).
menor valor → mayor prioridad.
- Los registros son de 32 bits como los demás, se implementa entonces que cada registro implementa 4 prioridades
- Ej: IPR0→ PRI[0..3], IPR1→ PRI[4..7], IPR{n}→ PRI{4n}..{4n+3}

0xE000E400 a 0xE000E4EF o NVIC->IPR[0..59]

```
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);  
                                // Establecer prioridad  
  
uint32_t NVIC_GetPriority(IRQn_Type IRQn); // Leer prioridad
```



Registros del NVIC

Registro de Activación por Software (STIR)

- Registro especial que permite disparar interrupciones externas simplemente escribiendo el número de interrupción deseado.
- A diferencia de otros registros del NVIC que requieren acceso privilegiado, el STIR puede configurarse para que tareas de usuario (no privilegiadas) puedan disparar interrupciones.

0xE000EF00 o NVIC->STIR

```
NVIC->STIR = 3; // esto tiene la misma función que poner en pendiente  
NVIC_SetPendingIRQ(Timer0_IRQn); // usando el reg. para pasar a pendiente
```



Registros del SCB

Dentro del bloque de control del sistema (*System Control Block*) hay registros que permiten una configuración avanzada del comportamiento de las excepciones en el Cortex-M3 y Cortex-M4.

El SCB es responsable de proporcionar información sobre la arquitectura del procesador y el control de las excepciones del sistema. Estos registros están mapeados en el espacio de control del sistema (SCS) y, por lo general, solo son accesibles en modo privilegiado.

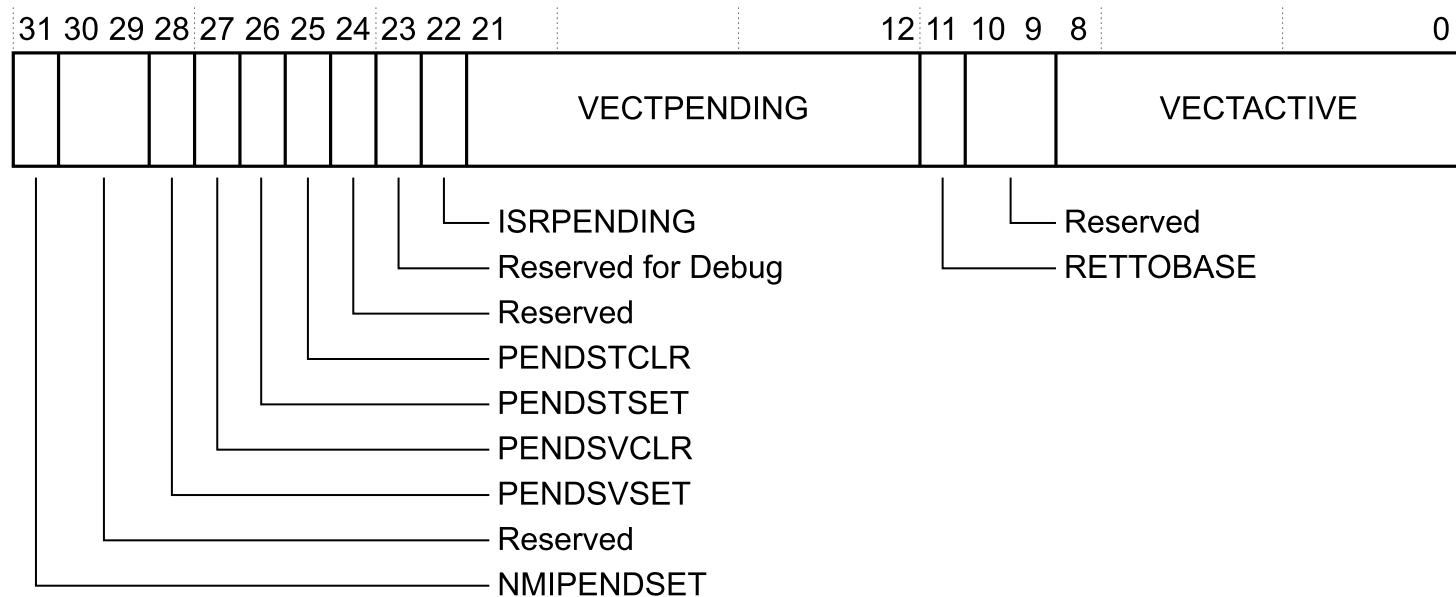
Registros del SCB

ICSR (Interrupt Control and State Register)

Este registro proporciona control sobre las excepciones NMI, SysTick y PendSV. Sus funciones principales incluyen:

- **Forzar excepciones:** Permite poner en estado pendiente excepciones como PendSV y SysTick por software.
- **Estado del sistema:** Muestra si hay interrupciones externas pendientes y el número de la excepción que se está ejecutando actualmente (VECTACTIVE)

(SCB->ICSR,
0xE000ED04)





Registros del SCB

VTOR (Vector Table Offset Register)

Este registro permite reubicar la tabla de vectores desde su dirección predeterminada (0x00000000) a otra ubicación en la memoria Flash o SRAM.

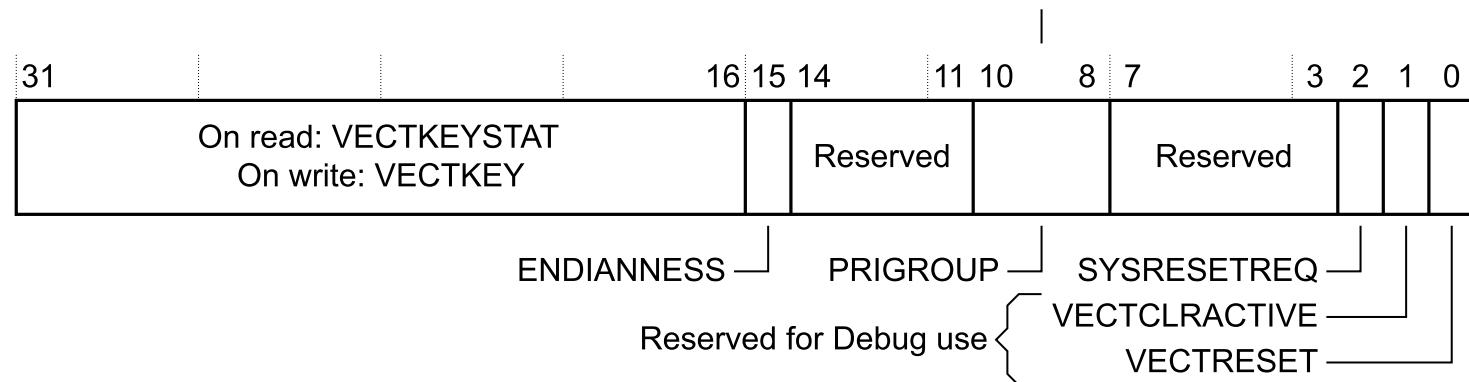
- Esto permite implementar bootloaders o para cambiar los manejadores de interrupción de forma dinámica durante la ejecución.
- La nueva dirección base debe estar alineada según el tamaño de la tabla de vectores (potencia de 2).

Registros del SCB

AIRCR (Application Interrupt and Reset Control Register)

Es uno de los registros más críticos para el control del procesador y requiere una clave de acceso de 16 bits (VECTKEY = 0x05FA) para realizar cualquier escritura. Sus funciones clave son:

- **Agrupación de prioridades (PRIGROUP):** Define cómo se divide el campo de prioridad.
- **Reset del sistema:** Permite solicitar un reinicio completo del microcontrolador mediante el bit SYSRESETREQ.
- **Orden de bits (Endianness):** Indica si el procesador está operando en modo little-endian o big-endian.





Registros del SCB

SHP (System Handler Priority Registers)

Estos registros se utilizan para configurar los niveles de prioridad de las excepciones del sistema (como SVC, PendSV y SysTick).
Funcionan de manera idéntica a los registros de prioridad del NVIC.

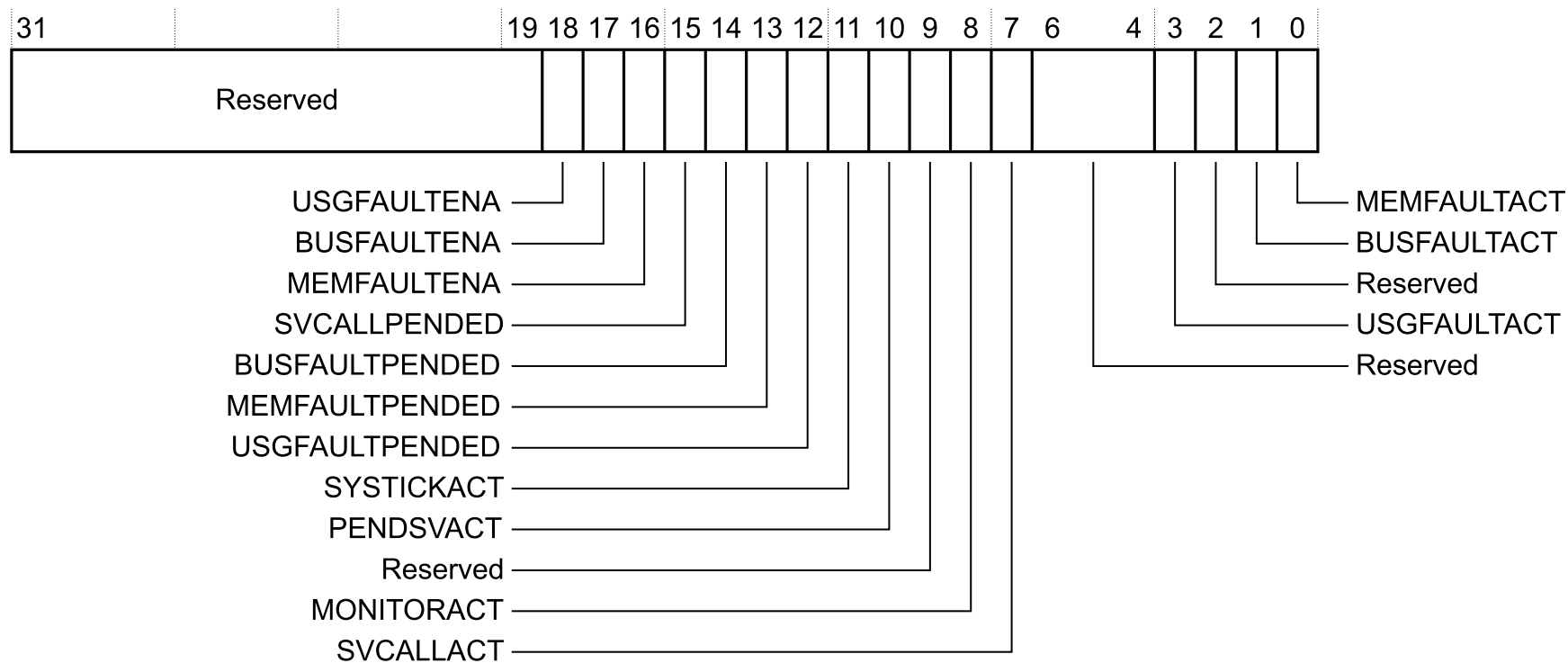
Nombre	Registro			
	Bit 31-24	Bit 23-16	Bit 15-8	Bit 7-0
SHPR1	Reservado	6-UsageFault	5-BusFault	4-MemManage
SHPR2	11-SVCall	Reservado		
SHPR3	15-SysTick	14-PendSV	Reservado	

Registros del SCB

SHCSR (System Handler Control and State Register)

Se encarga de la gestión de los manejadores de fallos específicos:

- Activar los ISR de MemManage Fault, Bus Fault y Usage Fault.
- Indica si estos fallos está activo o en estado pendiente.
- Si un fallo ocurre y no está habilitado se escalará a HardFault





Registros especiales para el enmascaramiento de excepciones o interrupciones

- **PRIMASK:** Cuando se activa, bloquea todas las excepciones e interrupciones que tienen prioridad programable elevando la prioridad a nivel 0 (solo quedan HardFault y NMI)
CMSIS: `__enable_irq()` y `__disable_irq()`
- **FAULTMASK:** eleva a -1 la prioridad, quedando solo NMI activa
CMSIS: `__enable_fault_irq()` y `__disable_fault_irq()`
- **BASEPRI:** Permite un enmascaramiento selectivo basado en el nivel de prioridad.
Valor 0: desactivado
Valor N: permite interrupciones con prioridad menor a N
CMSIS: `__set_BASEPRI(N)` y `__get_BASEPRI()`



Ejemplos de procedimientos para la configuración de interrupciones

Para el procedimiento directo, donde el vector se encuentra en la posición 0x0. Los pasos esenciales son:

- **Configurar la prioridad (Opcional).**
- **Habilitación en el periférico:** Se configura el periférico para que genere la señal de solicitud de interrupción.
- **Habilitación en el NVIC:** Se utiliza la función CMSIS `NVIC_EnableIRQ(IRQn)` para permitir que el controlador de interrupciones acepte la solicitud del periférico.
- **Nombre del manejador (ISR):** La función de interrupción en el código C de coincidir con el nombre definido en la tabla de vectores del archivo de inicio (startup code).



Ejemplos de procedimientos para la configuración de interrupciones

```
// Configurar el grupo a 5
NVIC_SetPriorityGrouping(5);

// Configurar la prioridad del Timer0_IRQn en 0xC0 (4 bit priority)
NVIC_SetPriority(Timer0_IRQn, 0xC); //la func corre de 0xC a 0xC0

// Habilitar la interrupción del Timer 0 en la NVIC
NVIC_EnableIRQ(Timer0_IRQn);
```



¿ Preguntas ?